

Rapport AP4A

FRAISSE Tom

Semestre d'Automne 2024

Table des matières

1	Introduction	2
1.1	Mise en contexte du sujet	2
1.2	Fonctionnement global	2
1.3	Outils de développement	3
2	Choix d'implémentation	4
2.1	Classe Server	4
2.2	Classe template Sensor	4
2.3	Classe Scheduler	5
2.4	Gestion des threads	5
3	Interface graphique avec QT	7
3.1	Aspect visuel de la fenêtre	7
3.2	liaison des signals entre les classes	8
4	Exécution	9
4.1	Console	9
4.2	QT	10
5	Améliorations	11
6	Conclusion	12
7	Annexes	13
7.1	Diagramme UML	13
7.2	Diagramme UML version Qt	14

1 Introduction

1.1 Mise en contexte du sujet

Le but initial de ce projet est de réaliser un **simulateur**, permettant de modéliser différents types de **capteurs** d'un environnement.

Ce simulateur est composé de :

- différents types de capteurs, tous capturant des données différentes.
 - Capteur de température : Float
 - Capteur d'humidité : Float
 - Capteur de son : Integer
 - Capteur de lumière : Boolean
- D'un serveur qui reçoit les données des différents capteurs pour ensuite les stocker sous forme de fichiers de logs et aussi les afficher directement dans la console.
- Et d'un scheduler qui joue le rôle d'ordonnanceur et envoie les instructions de lectures aux différents capteurs.

Premièrement, avant de commencer à travailler sur l'élaboration de ce projet, j'ai établi un cahier des charges de l'ensemble des attentes de ce projet dans un fichier markdown.

Ce qui m'a permis de fixer les objectifs de chacune des classes de ce projet.

1.2 Fonctionnement global

Lors de l'implémentation des premières fonctions de la classe serveur, je me suis rendu compte que je serais plus efficace avec une vue plus globale du fonctionnement du simulateur.

J'ai alors décidé d'établir un premier schéma resumant le fonctionnement global de ce simulateur.

En plus de cela, j'ai pu affiner un diagramme **UML** au fur et à mesure de l'implémentation de mes classes en partant de ce schéma de fonctionnement global. (Disponible en Annexe 1)

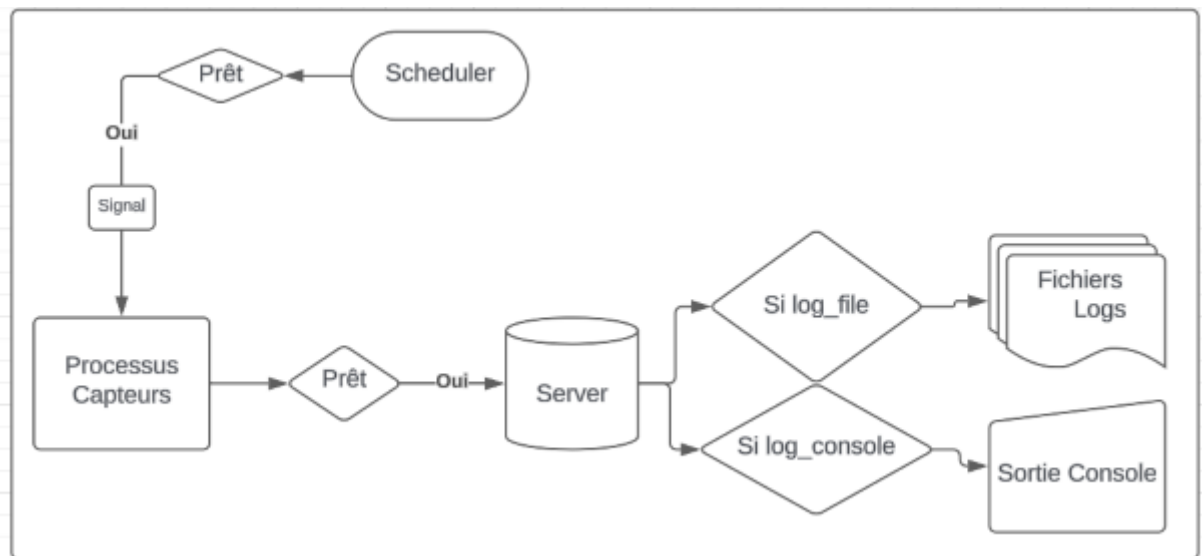


FIGURE 1.1 – Fonctionnement du simulateur

1.3 Outils de développement

Dans un premier temps j'ai choisi d'utiliser l'éditeur de code **CLion** avec un fichier **CMake** afin d'établir la base de mon projet et non un fichier **.pro** avec **QTCreator**.

De plus, ce projet a été développé principalement sous **Window** et avec une machine virtuelle sous **Ubuntu** afin d'utiliser la commande **valgrind** (l'utilisation de **Git** m'a permis de transmettre efficacement les fichiers entre les deux machines)

2 Choix d'implémentation

Cette deuxième partie explique les choix qui ont été pris durant l'implémentation de ce simulateur. Notamment vis-à-vis de la gestion des transmissions entre les capteurs et le server. Mais encore avec la gestion des threads émis par le Scheduler

2.1 Classe Server

Initialement, le Server doit être capable de récupérer les informations données par les capteurs pour ensuite les stocker ou les afficher.

Afin de gérer le transfert de données entre les capteurs et le server j'ai décidé d'utiliser un **tuple** contenant le message des différents capteurs directement transmis au server en surchargeant l'opérateur d'entrée. Il est important de noter que l'utilisation de fonction template a été nécessaire pour gérer la généricité du type de données transmises par les capteurs.

Le server peut ensuite stocker les données dans des fichiers textes (pour chacun des capteurs) et dans la console. Pour mettre en place un rôle plus actif du server, j'ai fixé un intervalle fixe qui permet au serveur de stocker les données selon cet intervalle et non dès qu'il reçoit un message.

2.2 Classe template Sensor

Pour cette classe j'ai décidé d'utiliser une classe abstraite afin de pouvoir créer les 4 types de capteurs grâce à un héritage hiérarchique.

Chaque capteur est créé avec un identifiant unique grâce à l'utilisation d'une fonction de hash avec la librairie `<hash>`. De plus, chaque capteur possède un intervalle de lecture propre aléatoire grâce à l'utilisation d'une distribution uniforme avec la librairie `<random>`.

Comme évoqué, cette classe possède une méthode virtuelle pure qui génère

une valeur aléatoire selon le type du capteur avec l'utilisation de différentes distributions de la librairie `<random>`.

De plus elle possède deux autres méthodes virtuelles pure, l'une qui envoie les données au serveur et l'autre qui permet de récupérer un pointeur unique (`unique_ptr`).

Enfin, afin de gérer efficacement la mémoire, j'ai décidé d'utiliser un pointeur intelligent partagé (`std::shared_ptr<Server*>`) afin d'associer un serveur à chaque capteur. Cela me permet d'éviter de gérer la mémoire lors de la destruction de mes capteurs.

2.3 Classe Scheduler

L'implémentation de cette dernière classe a été la plus compliquée selon moi car j'ai souhaité que le Scheduler crée plusieurs threads. De plus comme j'ai donné un rôle propre au Server, il m'a fallu lui allouer un thread.

Globalement, le Scheduler va ici gérer une liste de capteur et un serveur. Lors du début de la simulation, le programme crée un thread pour le serveur afin de vérifier s'il doit stocker les données, et un autre thread afin de transmettre les signaux à chacun des capteurs de la liste.

2.4 Gestion des threads

La figure ci-dessous présente le fonctionnement dynamique du Scheduler et plus précisément la création des Threads lors de l'appel de la fonction `start()`.

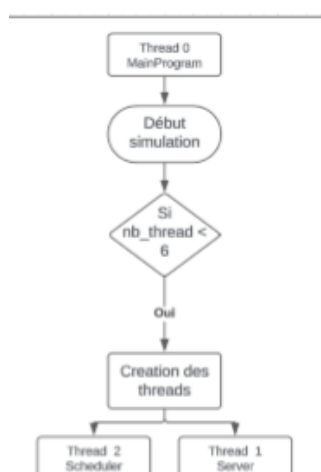
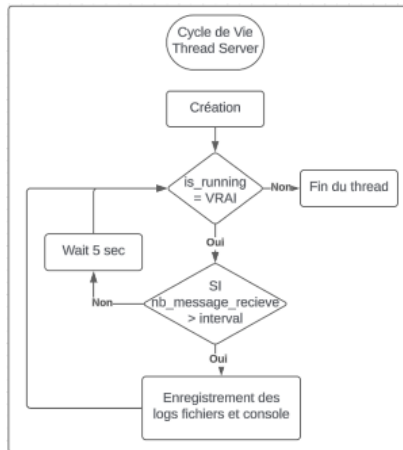
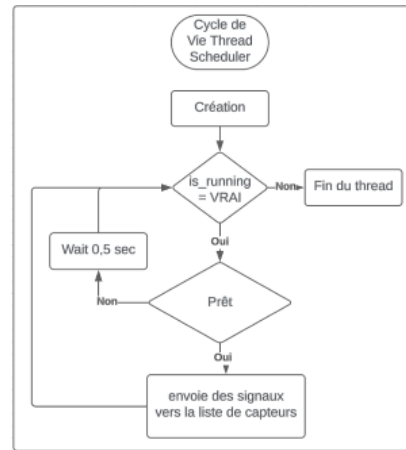


FIGURE 2.1 – Création des threads lors du début de la simulation

Les deux figures ci-dessous présentent quant à elles le cycle de vie des différents threads créés par le Scheduler. Simplement les deux threads effectuent leurs tâches distinctes tant que le Scheduler est en mode simulation.



(a) Thread lié au serveur



(b) Thread lié au scheduler

FIGURE 2.2 – Cycles de vies des Thread dans le Server et dans le Scheduler

3 Interface graphique avec QT

Comme il était possible d'utiliser Qt, j'ai saisi cette occasion afin de pouvoir découvrir le fonctionnement de Qt et la création d'une interface graphique simple avec cette API.

Dans un premier temps, j'ai développé l'aspect général de l'interface avec différents widgets pour ensuite faire le lien entre ces widgets et les signaux donnés par l'utilisateur grâce aux signaux.

3.1 Aspect visuel de la fenêtre

Premièrement, j'ai établi une esquisse de mon interface afin de visualiser l'agencement des différents widgets dans ma fenêtre. Pour ensuite, écrire le

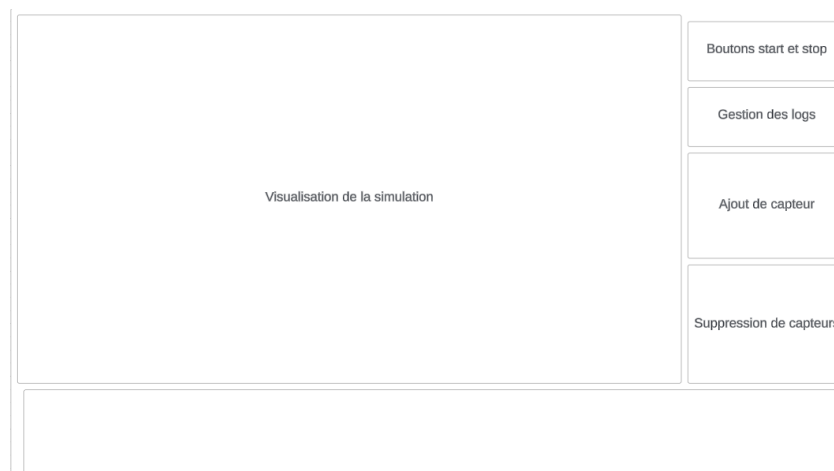


FIGURE 3.1 – Agencement global de la fenêtre

code nécessaire afin d'afficher les différents widgets (différents boutons / Zone de texte...) dans ma fenêtre. Une des notions importantes a été la gestion de Layout (Horizontale/ Verticale) qui regroupe différents widgets et/ou items selon un ordre précis.

3.2 liaison des signals entre les classes

Une fois l'interface implémentée, il était nécessaire de donner un comportement précis aux différents signaux donnés par l'utilisateur.

Pour effectuer cela, j'ai lié différents certains signaux à mes widgets en utilisant la fonction **connect()**.

De plus il a été nécessaire de légèrement modifier les classes **scheduler** et **server** afin que celles-ci aient la possibilité de transmettre des messages à ma fenêtre graphique (notamment à la console).

Après cela, j'ai pu lier la logique de mon simulateur initial, en transmettant la logique de simulation dans cet environnement graphique géré par l'utilisateur.

Enfin, j'ai pu vérifier le bon fonctionnement de mon application en utilisant les outils de débogage fournis par Qt, notamment avec la détection de fuite de mémoire (Memcheck).

4 Exécution

4.1 Console

```
PS C:\Users\tomfr\Desktop\Cursus Ingénieur\INFO\INFO1\AP4A\TP\Debug> ./TP -h
Usage : C:\Users\tomfr\Desktop\Cursus Ingénieur\INFO\INFO1\AP4A\TP\Debug\TP.e

Options : -h, --help      Print this help manual.
          -v, --version    Print program's version and compilation date.
          -l, --log        Turn on writing on log file.
          -c, --console    Turn on console writing.
```

FIGURE 4.1 – Choix des options

```
---
Bienvenue sur le simulateur IOT
---
Ajout de Capteurs dans la simulation
Donner le nom du capteur : Test1
Choisir le type de Capteur : T: temperature - H: humidite - S: son - L: lumiere
T
Ajouter un autre capteur (Y/N) ? |
```

FIGURE 4.2 – Ajout de capteurs dans la simulation

```
Tue Oct 15 11:04:46 2024 4880857654915686373 Temperature Test1 15.4419
Tue Oct 15 11:04:51 2024 8157993118037180760 Humidity tEST3 32.0893
Tue Oct 15 11:04:51 2024 4880857654915686373 Temperature Test1 17.9364
Tue Oct 15 11:04:56 2024 8157993118037180760 Humidity tEST3 63.7424
Tue Oct 15 11:04:56 2024 4880857654915686373 Temperature Test1 18.1081
Tue Oct 15 11:05:01 2024 8157993118037180760 Humidity tEST3 93.1724
Tue Oct 15 11:05:01 2024 4880857654915686373 Temperature Test1 9.81203
Tue Oct 15 11:04:41 2024 7273689483458078860 Sound test2 72
Tue Oct 15 11:04:46 2024 7273689483458078860 Sound test2 73
```

FIGURE 4.3 – Console log output

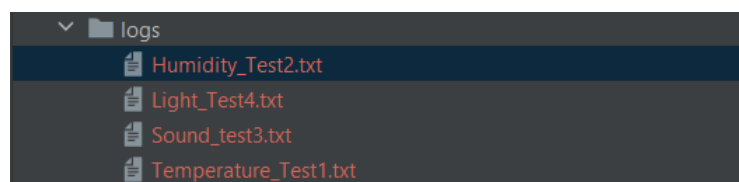


FIGURE 4.4 – Arborescence des fichiers de logs

4.2 QT



FIGURE 4.5 – Interface Qt

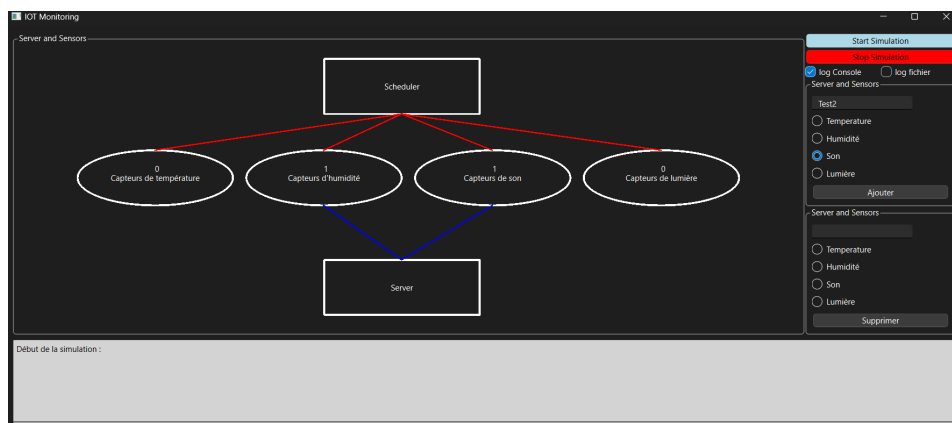


FIGURE 4.6 – Widget simulation en action

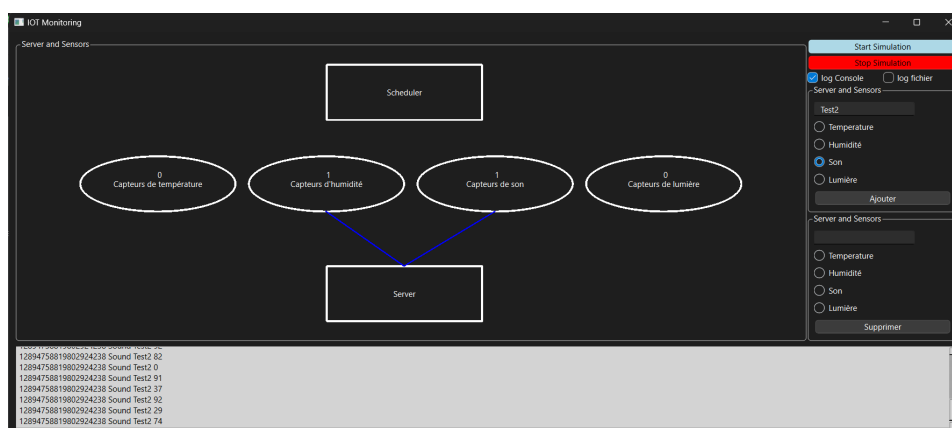


FIGURE 4.7 – Console Output

5 Améliorations

Ce projet propose de nombreuses pistes d'améliorations :

- Mise en place d'un contrôle total des capteurs à l'utilisateur par une gestion avancée des propriétés de chaque capteur (intervalles de lectures, Extremums...)
- Filtrage avancé des capteurs activés. Ici l'utilisateur pourrait étudier spécifiquement certains capteurs selon ses besoins
- Possibilité de gérer plusieurs parcs de capteurs selon différentes spécificités (Localisation, climat, environnement). On aurait une liste de Scheduler qui gère chacun un parc de capteurs.
- Production de données évolutive : le serveur serait capable de travailler avec une base de données. Afin de fournir à l'utilisateur des moyennes pour chacun des types de capteurs. Mais aussi de présenter les différentes fluctuations au cours du temps.

6 Conclusion

Ce projet a été pour moi l'occasion de mettre en application les différentes notions de la programmation orientée objet. Même si mon application est encore perfectible, j'ai pu intégrer les principes de base de la conception objet dans ce projet.

De plus, j'ai appris à utiliser d'autres notions comme l'utilisation des pointeurs intelligents et la bibliothèque random.

Enfin j'ai découvert l'environnement de développement Qt et les différentes techniques de base afin de créer une application 2D simple.

7 Annexes

Les diagrammes UML ci-dessous sont disponibles sous fichier .astah

7.1 Diagramme UML

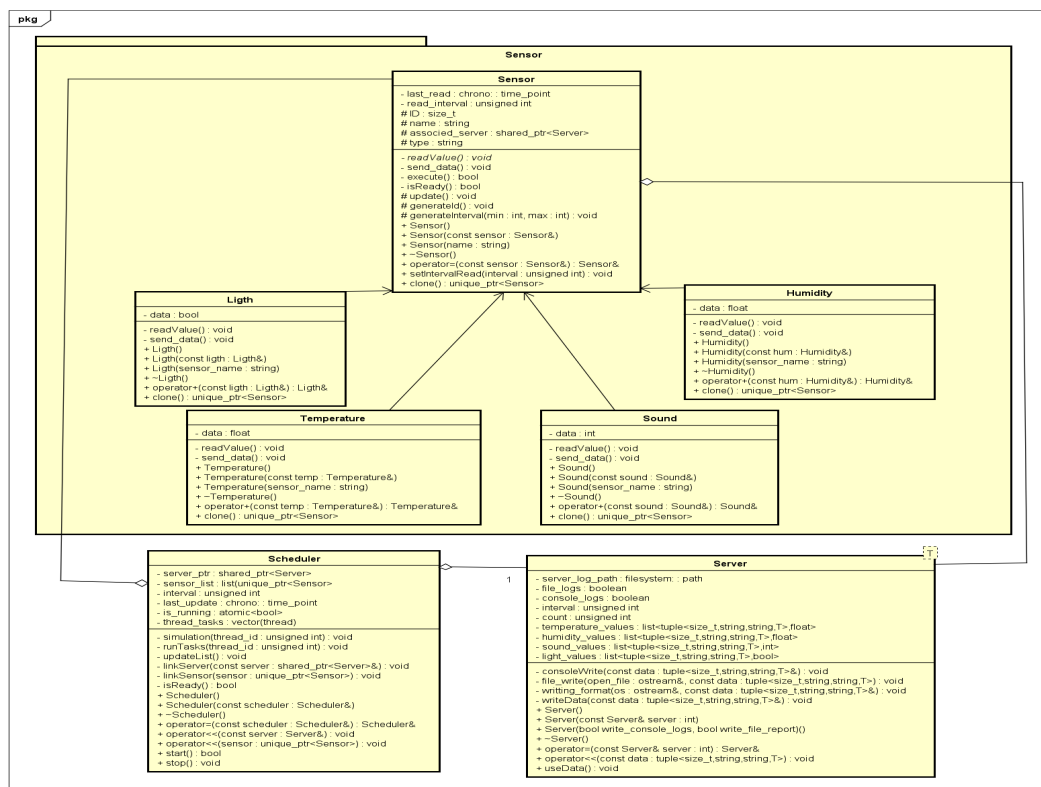


FIGURE 7.1 – Diagramme UML - Console

7.2 Diagramme UML version Qt

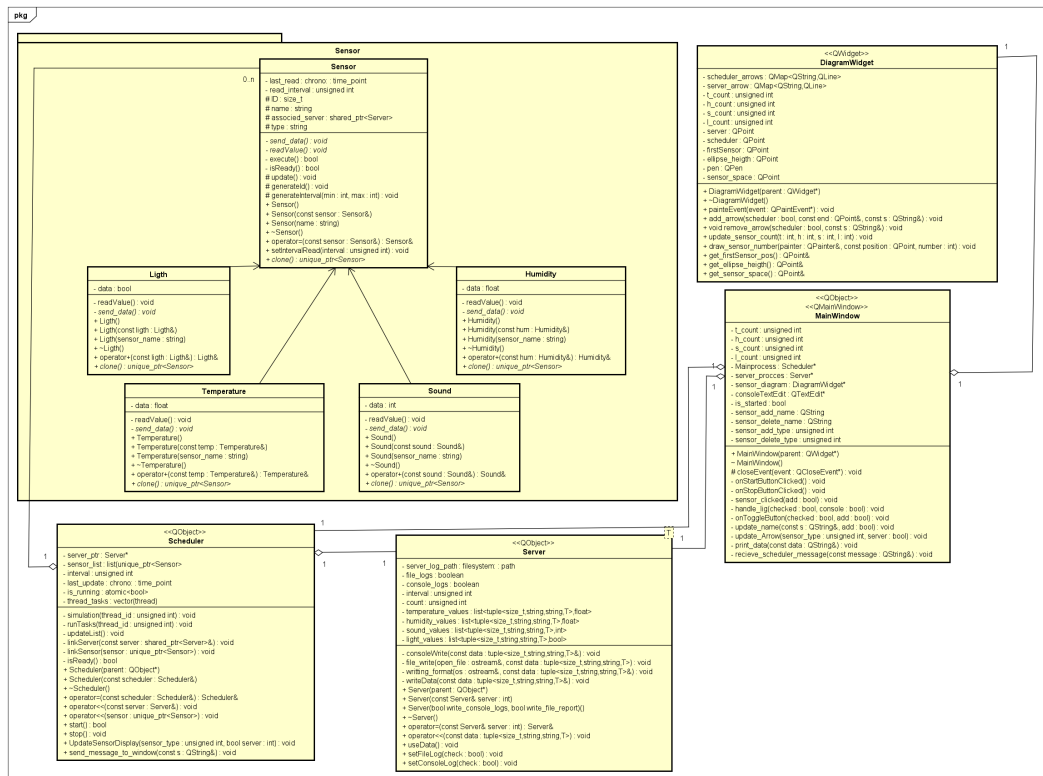


FIGURE 7.2 – Diagramme UML - QT